C PROGRAMMING

Lesson 3 — Format Specifiers

Based on: Bro Code C Tutorial Series

1. What are Format Specifiers?

Format specifiers are special tokens used inside printf() to tell C how to display a variable. They always start with a percent sign % followed by a character that represents the data type.

```
int age = 25;
printf("Age: %d", age);   // %d is the format specifier for int

// Output: Age: 25
```

> **Rule:** The format specifier must match the data type of the variable. Using the wrong one causes incorrect output or errors.

2. General Structure of a Format Specifier

A format specifier can have optional parts that control how the output looks:

```
%[flags][width][.precision][length]specifier

// Examples:
//  %d        -->  basic integer
//  %8d       -->  integer, minimum width of 8
//  %-8d      -->  left-justified, width 8
//  %08d      -->  zero-padded, width 8
//  %+d       -->  always show sign (+ or -)
//  %.2f      -->  float with 2 decimal places
//  %8.2f     -->  width 8, 2 decimal places
```

You do not need to use all parts — start simple with just the specifier, and add modifiers when needed.

3. Common Format Specifiers

Data Type	Specifier	Example Code	Output
int	`%d`	`printf("%d", 25);`	`25`
float	`%f`	`printf("%f", 19.99);`	`19.990000`
double	`%lf`	`printf("%lf", 3.14159);`	`3.141590`
char	`%c`	`printf("%c", 'A');`	`A`
string	`%s`	`printf("%s", "Hello");`	`Hello`

> **Double vs Float:** Use %lf for double. While %f works for output, %lf is preferred for consistency — especially when you learn scanf() later.

4. Optional Modifiers

Width — Minimum character space

Sets the minimum number of characters printed. If the value is shorter, spaces are added on the left by default.

```
printf("%4d\n", 1);    //  Output:    1  (3 spaces + 1)
printf("%4d\n", 10);   //  Output:   10  (2 spaces + 10)
printf("%4d\n", 100);  //  Output:  100  (1 space  + 100)
printf("%4d\n", 1000); //  Output: 1000  (no padding needed)
```

Left Justification — Minus flag -

By default, values are right-justified. Add a - before the width to left-justify instead.

```
printf("%-4d|\n", 1);    //  Output: 1   |
printf("%-4d|\n", 10);   //  Output: 10  |
printf("%-4d|\n", 100);  //  Output: 100 |
```

Zero Padding — Zero flag 0

Pads the number with leading zeros instead of spaces.

```
printf("%04d\n", 1);    //  Output: 0001
printf("%04d\n", 10);   //  Output: 0010
printf("%04d\n", 100);  //  Output: 0100
```

Sign Flag — Plus flag +

Forces the sign to always be shown — even for positive numbers.

```
printf("%+d\n", 25);   //  Output: +25
printf("%+d\n", -25);  //  Output: -25
```

Precision — Decimal places for floats

Controls how many digits appear after the decimal point. Syntax: %.nf where n is the number of decimal places.

```
float price = 19.99;
printf("%f\n",   price);  //  Output: 19.990000  (default: 6 places)
printf("%.2f\n", price);  //  Output: 19.99
printf("%.0f\n", price);  //  Output: 20  (rounded)
```

> **Tip:** Use %.2f for money/currency — it always shows exactly 2 decimal places.

Combining Modifiers

You can combine width, precision, and flags together in one specifier.

```
printf("%+8.2f\n", 19.99);   //  Output:   +19.99  (width 8, 2
decimals, sign)
printf("%+8.2f\n", -100.0);  //  Output:  -100.00
```

5. Practice Problem

Practice: Shop Receipt

Write a C program that prints a simple shop receipt. This will test your ability to use format specifiers and modifiers together.

Requirements:

- Declare 3 items with a name (string) and price (float)
- Print each item with the name left-justified in 15 characters and the price formatted to 2 decimal places
- Print a total at the bottom with a + sign and 2 decimal places

Expected Output (example)

```
Apple          $  1.50
Bread          $  2.99
Milk           $  1.25
----------------------
Total          $ +5.74
```

Hint: Use %-15s for left-justified item names and %6.2f for prices. Combine them in one printf() line.